



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2233 — Programación Avanzada Programa de curso 2026-2

Clases: Jueves, módulos 5 y 6
Formato: Presencial
Requisitos: IIC1103 — Introducción a la Programación
Sitio web: github.com/IIC2233/Syllabus/

Sección	Profesor	Correo	Ayudantía
1	Jaime Middleton	jamiddleton@uc.cl	Martes, módulo 4
2	Rocío Ladrón de Guevara	rociohgj@uc.cl	Martes, módulo 4
3	Antonio López	alopez7@uc.cl	Martes, módulo 5
4	Tamara Vidal	ttvidal@uc.cl	Martes, módulo 5
5	Daniela Concha	daconcha@uc.cl	Martes, módulo 6

Introducción

El desarrollo de la computación ha hecho que estemos rodeados de dispositivos que ejecutan programas computacionales todo el tiempo. Un aspecto importante para ser partícipes del desarrollo de estas herramientas es comprender su funcionamiento y cómo construir programas simples y *software* más complejo.

Este curso profundiza aspectos de la programación que se introdujeron en cursos anteriores, en particular, estructuras y técnicas de diseño que permiten construir herramientas más complejas y prácticas.

Objetivo General y Competencias

A lo largo de este curso, el estudiante desarrollará técnicas para diseñar, implementar, ejecutar y evaluar herramientas de *software* que resuelvan problemas algorítmicos a partir de especificaciones detalladas. El estudiante será capaz de desarrollar construcciones avanzadas de programación orientada a objetos y estructuras de datos fundamentales, construir código robusto, construir interfaces gráficas, y utilizar correctamente conceptos como *threading*, serialización y paso de mensajes.

Al finalizar el curso, el estudiante será capaz de:

1. **Aplicar técnicas básicas de mantención de código**, incluyendo el uso de guías de estilo, modularización y sistemas de manejo de versiones (Git).
2. **Inferir un modelo de objetos** para resolver problemas realistas e **implementar esta solución** usando técnicas de programación orientada a objetos (OOP).
3. **Construir objetos iterables** para resolver problemas de programación.
4. **Utilizar el Paradigma de Programación Funcional** para procesar datos.
5. **Aplicar el concepto de *threading*** para la modelación de problemas de concurrencia.
6. **Construir interfaces gráficas** funcionales utilizando módulos apropiados.

7. **Utilizar técnicas para construir código robusto** en un programa, como las formas de manejo de excepciones, entre otros.
8. **Utilizar el concepto de serialización** para construir codificadores y decodificadores.
9. **Aplicar infraestructuras y técnicas básicas** para crear y mantener **aplicaciones distribuidas**, asegurando el intercambio de datos y su persistencia, entre otros.
10. **Conocer** diferentes herramientas, prácticas y/o algoritmos actuales en el contexto de desarrollo de *software*.

Contenidos

Fundamentos de programación

- **Estructuras de datos básicas:** listas, tuplas, *named tuples*, *stacks*, colas, diccionarios, *sets*, empaquetamiento de datos.
- **Programación orientada a objetos:** clases, objetos, herencia, polimorfismo, clases abstractas.
- **Iterables:** objetos iterables, objetos iteradores, generadores y listas ligadas.
- **Programación funcional:** paradigma, funciones de mapeo, filtro y reducción.
- **Manejo de errores:** análisis de errores, tipos de excepciones, levantamiento y control de excepciones.

Herramientas de programación

- **Técnicas básicas de mantención de código:** concepto y uso de herramientas de sistemas de manejo de versiones, uso de guías de estilo y modularización.
- **Threading:** concepto de pseudo-parallelismo, concurrencia, creación y sincronización de *threads*.
- **Interfaces gráficas:** introducción a las interfaces gráficas usando la librería PyQt. Uso de señales para la comunicación entre *backend* y *frontend*.
- **I/O:** *bytes*, serialización binaria, serialización en formato JSON.
- **Networking:** protocolos y arquitecturas de comunicación, *sockets*, modelo cliente-servidor e intercambio de información, sistemas distribuidos, entre otros.
- **Aplicaciones:** *Web services*, consumo y generación de API, expresiones regulares, entre otros.

Metodología

El curso seguirá una metodología *blended* en la cual se utilizará el modelo de clase invertida (*flipped classroom*) con sesiones expositivas en clases, sesiones de apoyo en ayudantía, evaluaciones escritas, controles y actividades de programación en clase, y ejercicios propuestos de programación.

La modalidad de la clase de cada semana será *flipped classroom*. En estas, se asume que los estudiantes ya estudiaron los contenidos de manera previa a la clase, para luego validar su comprensión y aplicación mediante evaluaciones de cátedra. Para este fin, el material de estudio se encontrará disponible en el sitio del curso (al menos) desde el viernes anterior a la clase. El material de estudio consiste en apuntes donde se describen detalladamente los tópicos y ejercicios propuestos de programación con las que el estudiante podrá desarrollar las competencias asociadas a cada contenido.

En el horario de cátedra, el profesor hará una sesión de consultas sobre los contenidos de la semana. Luego, se realizará un control de entrada de 10 minutos. Posteriormente, se destinarán 2 horas para desarrollar la actividad publicada durante la misma clase en la cual se aplican los nuevos conocimientos. Tanto el profesor como los

ayudantes del curso estarán presentes durante toda la duración de esta actividad para resolver dudas y guiar a los estudiantes. Finalmente, durante los últimos 10 minutos de la cátedra, se realizará un cierre de la actividad.

El curso contará con ayudantías semanales con el fin de aplicar los contenidos antes de la clases. Es importante notar que las sesiones de reforzamiento de contenidos también asumen revisión previa del material y no están orientadas a ser un reemplazo de estudio previo, sino que son una instancia de resolución de dudas previa a la clase.

Evaluación

La evaluación será efectuada mediante dos (2) instrumentos: evaluaciones escritas, y evaluaciones en cátedra. Las fechas y horarios son los siguientes:

Evaluaciones Escritas		
	Fecha	Horario
I1	10 de septiembre	14:50 a 17:20
I2	22 de octubre	14:50 a 17:20
Examen	26 de noviembre	14:50 a 17:20

(1) Evaluaciones Escritas (EE) [60 %] El curso realizará tres (3) evaluaciones presenciales escritas a desarrollar con lápiz y papel: Interrogación 1, Interrogación 2 y Examen. Estas evalúan que cada estudiante interiorizó correctamente los contenidos básicos del curso, es capaz de aplicar los contenidos en diversos problemas, y es capaz de identificar posibles errores de código junto con proponer mejoras.

Alcance y reglas de evaluación:

- **Carácter acumulativo:** la materia evaluada es acumulativa durante el semestre.
- **Materia considerada:** las preguntas podrán abarcar todo lo relacionado al curso, incluyendo contenidos de clases y actividades.

Cálculo de nota: La nota de las evaluaciones escritas se calcula como el promedio de todas las evaluaciones, es decir:

$$EE = \frac{3 \times \mathbf{I1} + 3 \times \mathbf{I2} + 4 \times \mathbf{EXAMEN}}{10}$$

Uno de los requisitos de aprobación del curso es que **EE** debe ser mayor o igual a 3,95.

(2) Evaluaciones de Cátedra (EC) [40 %] Para evaluar el método *flipped classroom*, se realizarán múltiples evaluaciones en el horario de cátedra. Las fechas se detallan en el calendario del curso. Cada evaluación consistirán de un **Control de entrada (C)** y una **Actividades de programación (AC)**, los cuales serán explicadas con mayor detalle en sus secciones correspondientes.

Asistencia: Todas las evaluaciones de cátedra se desarrollarán de forma presencial durante la cátedra del curso. Durante la evaluación se pasará asistencia tanto al inicio como al final de la evaluación, el quedar ausente en el listado de entrada o salida implicará que se asignen 0 puntos tanto en el control de entrada como la actividad de programación. Dada la naturaleza de esta evaluación que contempla tiempo en clases para su desarrollo, y para fomentar la metodología *flipped classroom*, no se podrá optar a evaluaciones de cátedra recuperativas.

Cálculo de nota: La nota de evaluaciones de cátedra (**EC**) se calcula a partir de **puntos de cátedra (PC)** que se obtienen al resolver los controles de entrada y actividades de programación. Cada control de entrada otorgará un (1) **punto de cátedra (PC)** y cada actividad de programación otorgará cinco (5) **puntos de cátedra (PC)**.

La nota de evaluaciones de cátedra se calculará a partir de la cantidad total de evaluaciones de cátedra que se realicen durante el semestre mediante la siguiente fórmula:

$$EC = \min \left(7,0; \frac{PC}{Cantidad\ EC - 2} + 1 \right)$$

Dado que no se necesita el puntaje máximo para obtener nota 7,0; los puntos restantes permiten compensar inasistencias.

Controles de entrada (C) Para fomentar la metodología de *flipped classroom*, previo a cada actividad de programación se aplicará un **control de entrada** que el estudiante deberá resolver de forma individual. Dado el carácter acumulativo del curso, cada control de entrada incluye todo el contenido visto hasta el momento, a menos que se explicita lo contrario, pero enfocándose en el contenido de la semana.

Publicación y entrega: El control se realizará de forma presencial al inicio de la clase, de 15:00 a 15:10.

Asignación de puntaje: Cada control de entrada otorgará un (1) punto de cátedra (**PC**), lo que aporta a la nota de Evaluaciones de Cátedra (**EC**). Se definirá en cada control los requerimientos necesarios para obtener la totalidad de puntos asociados a esta.

Actividades de programación (AC) Con el fin de controlar que se esté usando la metodología de *flipped classroom*, periódicamente se realizarán actividades que los estudiantes deberán resolver de forma individual durante la clase. Dado el carácter acumulativo del curso, cada actividad incluye todo el contenido visto hasta el momento, a menos que se explicita lo contrario, pero enfocándose en el contenido de la semana.

Publicación y entrega: Se subirá la actividad durante el periodo de la clase. Esta debe ser entregada como máximo a las 17:10 del mismo día. Las actividades de programación se entregan **únicamente** a través de un repositorio privado y personal del estudiante. Este se alojará en la plataforma **GitHub** y será provisto por el equipo docente a cada estudiante. **GitHub es el medio de entrega oficial y único del curso, donde cada actividad especificará el nombre de la carpeta en que debe entregarse dentro del repositorio del estudiante.**

Asignación de puntaje: La corrección se realizará mediante una revisión manual. Cada actividad otorgará cuatro (4) puntos de cátedra (**PC**), lo que aporta a la nota de Cátedra (**C**). Se definirá en cada actividad los requerimientos necesarios para obtener la totalidad de puntos asociados a este.

Política de atraso e inasistencia.

- Evaluaciones de cátedra: **No se aceptarán controles de entrada ni actividades de programación entregados fuera de plazo.** A toda actividad o control no entregado se le asignará 0 puntos. Adicionalmente, durante cada evaluación de cátedra se pasará asistencia al inicio y final de la evaluación, el **quedar ausente en el listado de entrada o salida implicará que se asignen 0 puntos** tanto en el control de entrada como la actividad de programación. Finalmente, no se podrá optar a evaluaciones de cátedra recuperativas.
- Evaluaciones Escritas: Las evaluaciones escritas tendrán posibilidad de justificar máximo **una** (1) inasistencia durante el semestre. Se debe justificar ante la Dirección de Pregrado correspondiente, quienes entregarán una carta de inasistencia. Finalmente, es responsabilidad del estudiante enviar esta carta vía correo electrónico a la coordinación del curso. El plazo para justificar una inasistencia es de 5 días (lunes a domingo sin importar si es feriado) desde que inició la respectiva evaluación. En el caso de tener licencia médica, el tiempo comienza a correr desde el reintegro a la universidad. La recuperación de la evaluación justificada será discutida por el equipo docente.

Proceso de corrección. Luego de publicadas las puntajes de evaluaciones de cátedra y las notas de evaluaciones escritas, se dará un período de una semana para recibir solicitudes de corrección vía formulario. Cada solicitud debe estar debidamente justificada, y debe ser enviada por los canales que el curso disponga para este propósito.

Cabe destacar que en el proceso de corrección, el cuerpo docente puede revisar completamente el trabajo enviado y no sólo los ítems que se están apelando. Por lo cual, la nota puede subir, bajar o mantenerse si se considera que uno o más ítems no fueron corregidos como corresponde según la rúbrica y los contenidos aplicados.

Nota final y aprobación. Sea **NP** la nota de presentación del curso, que se calcula como:

$$NP = \frac{6 \times EE + 4 \times EC}{10}$$

Para aprobar el curso, el estudiante debe cumplir con:

- **NP** debe ser mayor o igual a 3,95.
- **EE** debe ser mayor o igual a 3,95.
- **EC** debe ser mayor o igual a 3,95.

La nota final del curso **NF** se calculará como:

$$NF = \begin{cases} NP & \text{si cumple todos los requisitos.} \\ \min(NP; 3,9) & \text{en otro caso.} \end{cases}$$

Todas las notas y promedios del curso serán calculadas con un redondeo a **dos decimales**, salvo la nota final del curso que se calculará con un redondeo a **un decimal**.

Integridad Académica

En este curso se espera que cada estudiante mantenga los más altos estándares de integridad académica. Toda evaluación (actividades, controles, evaluaciones escritas, y cualquier otro instrumento evaluativo del curso) debe reflejar el trabajo personal del estudiante y el cumplimiento de las instrucciones de cada entrega.

Normativa aplicable y sanciones. Cualquier situación de **falta a la integridad académica** detectada en alguna evaluación tendrá como **sanción un 1,1 final en el curso**, sin perjuicio de sanciones adicionales de acuerdo con la Política de Integridad Académica de la Escuela de Ingeniería y de la Universidad. Rige para este curso tanto la política de integridad académica del Departamento de Ciencia de la Computación (ver anexo) como el [Código de Honor de la Escuela de Ingeniería](#).

Conductas prohibidas (lista no exhaustiva). Se considera falta a la integridad académica, entre otras:

- Copiar total o parcialmente el trabajo de otra persona, o permitir que otra persona copie el propio.
- Compartir, solicitar, comprar, vender o intercambiar soluciones (o fragmentos) de evaluaciones actuales o pasadas del curso, incluyendo código, respuestas escritas, *tests*, archivos, reportes o explicaciones.
- Utilizar material no autorizado durante evaluaciones presenciales (apuntes, dispositivos, internet, repositorios, herramientas, etc.), o comunicarse con terceros durante una evaluación.
- Presentar como propio trabajo realizado por terceros (incluyendo estudiantes, exalumnos, tutores o servicios externos) sin la debida autorización y referencia.
- Plagio o ausencia de citación adecuada al reutilizar material externo (código, textos, ideas, diagramas, documentación, etc.).

- Manipular el proceso de evaluación: alterar archivos, contenido de los *commits*, metadatos, registros, o intentar evadir o alterar mecanismos de corrección o supervisión.
- Suplantación de identidad o rendición de evaluaciones por otra persona.

Uso de herramientas de IA. Las herramientas de asistencia basadas en IA (por ejemplo, ChatGPT, Copilot u otras herramientas similares) **solo están permitidas durante el estudio de contenidos**. Su uso está **expresamente prohibido** en **controles de entrada, actividades de programación, evaluaciones escritas (interrogaciones y examen) y cualquier otra evaluación** del curso, salvo que el equipo docente indique lo contrario de forma explícita y por escrito.

Uso de código de terceros y material público. Debido a la naturaleza de la disciplina en la que se enmarca el curso, se permite el uso de código escrito por una tercera parte **solo bajo ciertas condiciones y siempre correctamente referenciado**, indicando la fuente de donde se obtuvo.

- Se permite el uso de código encontrado en internet u otra fuente pública **siempre que su autor sea externo al semestre actual del curso** y que su publicación sea **previa a la liberación del enunciado** correspondiente. Su uso debe ser producto de un proceso de investigación, integración y adaptación (no copia directa sin comprensión).
- Se permite utilizar el código publicado en los repositorios oficiales del curso, incluyendo el presente en contenidos y ayudantías, respetando las instrucciones de cada evaluación.
- Se prohíbe estrictamente **compartir, reutilizar o utilizar** código correspondiente a evaluaciones del curso de **semestres anteriores** (propias o ajenas), o cualquier material que permita reconstruir soluciones evaluadas en dichos semestres. Esto será considerado una falta a la integridad académica. En cambio, **sí está permitido** utilizar material trabajado **durante el semestre en curso**, incluyendo código y recursos mostrados en clases, ayudantías, y trabajos desarrollados por el propio estudiante en el marco del curso, siempre respetando las instrucciones específicas de cada evaluación y citando fuentes externas cuando corresponda.

ANTE CUALQUIER DUDA SOBRE QUÉ ESTÁ PERMITIDO EN UNA EVALUACIÓN, EL ESTUDIANTE DEBE ASUMIR QUE **NO ESTÁ PERMITIDO** Y CONSULTAR AL EQUIPO DOCENTE ANTES DE ENTREGAR O RENDIR LA EVALUACIÓN.

Bibliografía

- K. Pichara, C. Pieringer. *Advanced Programming in Python*, disponible en [Amazon](#).

Anexo: Política de integridad académica del Departamento de Ciencia de la Computación

Los/as estudiantes de la Escuela de Ingeniería de la Pontificia Universidad Católica de Chile deben mantener un comportamiento acorde a la Declaración de Principios de la Universidad. En particular, se espera que mantengan altos estándares de honestidad académica. Cualquier acto deshonesto o fraude académico está prohibido; los/as estudiantes que incurran en este tipo de acciones se exponen a un Procedimiento Sumario. Es responsabilidad de cada estudiante conocer y respetar el documento sobre Integridad Académica publicado por la Dirección de Docencia de la Escuela de Ingeniería.

Específicamente, para los cursos del Departamento de Ciencia de la Computación, rige obligatoriamente la siguiente *política de integridad académica*.

Todo trabajo presentado por un/a estudiante para los efectos de la evaluación de un curso debe ser hecho por el/la estudiante. El/la estudiante es **responsable de originar el material entregado y de conocer íntegramente su contenido**. Asimismo, deberá respetar estrictamente las condiciones definidas por el curso para cada trabajo (por ejemplo, si el trabajo es individual, grupal, y si se permite o no el uso de material digital o de asistentes de inteligencia artificial). Por “trabajo” se entiende en general las interrogaciones escritas, las tareas de programación u otras, los trabajos de laboratorio, los proyectos, el examen, entre otros.

Si un/a estudiante incurre en una falta a la integridad académica, como, por ejemplo, entregando trabajo que no es propio, no comprendiendo íntegramente el trabajo entregado, o no cumpliendo con las condiciones establecidas por el curso, **obtendrá nota final 1.1 en el curso** y se solicitará a la Dirección de Pregrado de la Escuela de Ingeniería que no le permita retirar el curso de la carga académica semestral. En todos los casos, se informará a la Comisión de Integridad Académica de la Escuela de Ingeniería para que tome sanciones adicionales si lo estima pertinente.

En caso de utilización de fuentes digitales como Wikipedia o el uso de asistentes inteligentes como ChatGPT o Copilot, cuando se permita su uso, debe declararse de forma explícita el material o herramienta usada. En caso de uso de asistentes de inteligencia artificial, el profesor/a del curso puede solicitar que se entreguen respaldos sobre su utilización (por ejemplo, historial o *logs*).

Lo anterior se entiende como complemento al Reglamento del Estudiante de la Pontificia Universidad Católica de Chile (registrosacademicos.uc.cl/reglamentos/estudiantiles/). Por ello, es posible pedir a la Universidad la aplicación de sanciones adicionales especificadas en dicho reglamento.

Anexo: Compromiso del Código de Honor

Este curso suscribe el Código de Honor establecido por la Universidad, el que es vinculante. Todo trabajo evaluado en este curso debe ser propio. En caso que exista colaboración permitida con otros/as estudiantes, el trabajo deberá referenciar y atribuir correctamente dicha contribución a quien corresponda. Como estudiante es un deber conocer el Código de Honor (uc.cl/codigo-de-honor/)